Java OOP Laboratory Exercise: Campus Cafe Ordering System

Student name: Christoff Allen B. Jumapit

Section: _____

Date: November 5, 2026

Exercise title: Java OOP Laboratory Exercise: Campus Cafe Ordering System

Concept Explanation

Concept	Where and why it was used
If statement	Used in Pastry.computePrice() for the warming fee and in Meal.computePrice() for the upsize fee. The fee is added only when the condition is true.
If-else statement	Used in MenuItem.setBasePrice(), Beverage.computePrice(), Order.addItem(), and Order.computeTotal(). They check the price, size, item limit, and student discount.
Classes and objects	The program has 7 classes. CafeApp creates the objects (customers, items, and orders) and makes them work together.
Attributes and behaviors	Each class has fields (like basePrice or loyaltyPoints) and methods (like computePrice() or addPoints()). The fields hold the data and the methods do the work.
Object interaction	Order stores a Customer and three MenuItem objects. printReceipt() calls the items to get prices and updates the customer's loyalty points.
Java naming conventions	Classes use PascalCase, methods and variables use camelCase, and constants such as MAX_ITEMS use UPPER_SNAKE_CASE. This keeps the code easy to read.
Default constructor	MenuItem(), Beverage(), Pastry(), Meal(), and Customer() set starting values. CafeApp uses them to make Hot Chocolate, Cookie, and Beef Tapa.
Parameterized constructor	The three item classes, Customer, and Order take values when the object is created. This lets me create an object in one line.
Constructor overloading	Customer has three constructors: Customer(), Customer(String), and Customer(String, boolean). Each one handles a different amount of information.
public, private, protected	Fields are private, except name and basePrice in MenuItem, which are protected so the subclasses can use them. Methods are public so other classes can call them.
Getters, setters, data hiding	Other classes can reach the fields only through getters and setters. The setters check the data, for example they reject a negative price or an invalid size.
Method overloading	Customer.addPoints(int) and addPoints(double) have the same name but different parameters. Order.computeTotal() and computeTotal(double) do the same, and the second one also takes a voucher.
Method overriding	Beverage, Pastry, and Meal override computePrice() and getDescription() with @Override. Each type gives its own price and description.
Abstraction	MenuItem is abstract and has the abstract method computePrice(). It cannot be created directly, and every subclass must define its own pricing.

Screenshots of the Program Output

The screenshot below shows the output of java CafeApp, with both receipts, the fourth-item rejection, and the negative-price error.

[Insert screenshot of the actual program output here]

Challenges Encountered and How I Solved Them

- Three item fields, no ArrayList. I used item1, item2, and item3, and an if-else-if chain in addItem() to find the first empty slot. The itemCount field stops a fourth item.
- The voucher has no field in Order. The required attributes do not include a voucher, so I added a second printReceipt(double voucher) method. printReceipt() calls it with 0.
- Constructors must follow the validation rules. A constructor that sets a field directly can skip the setter's check. I called the setters inside the constructors and set safe starting values first.
- Loyalty points. The points use whole numbers, so I cast the result to int. 285.30 pesos gives 5 points and 205.00 pesos gives 4 points, which matches the sample output.